NVIDIA Cosmos, Explained in Depth

A beginner-to-intermediate guide to all four generations — Cosmos 1, 2, 2.5, and 3 — with diagrams, model sizes, data, inference speed, and the pretraining + post-training (SFT / GRPO) recipes.

What is a “World Foundation Model” (WFM)?

A **language** model learns from text and predicts the next *word*. A **World Foundation Model** learns from video and predicts what happens next in the *physical world* — how things move, collide, and behave. NVIDIA **Cosmos** is a family of WFMs for **Physical AI** (robots, self-driving cars): instead of testing a robot millions of times in reality, engineers let Cosmos **imagine** realistic video of what would happen and train on that.

> *Mental model: Cosmos is a “dream engine” for machines. Each generation dreams with higher quality, runs faster, understands your request better, and — by Cosmos 3 — even hears sound and decides physical actions.*

Mini-glossary (so the rest makes sense)

Tokenizer	Compresses video into a small set of numbers (“tokens”) the model can handle, and decompresses them back to pixels.
Diffusion	Starts from random noise and repeatedly “cleans” it into a video. High quality, but many steps = slower.
Autoregressive (GPT-style)	Generates the video one chunk at a time, like typing word by word. Fast, good for real-time.
Flow-based	A cleaner, more direct cousin of diffusion: learns a straight “flow” from noise to data, often with fewer steps.
DiT	“Diffusion Transformer” — the Transformer architecture that powers the diffusion engine.
Mixture-of-Transformers	Several specialized Transformers in one model (e.g. one that *reasons* + one that *generates*).
Pretraining	Learning general knowledge from a huge, broad dataset.
SFT	Supervised Fine-Tuning: teach the model on curated question→answer examples after pretraining.
GRPO	An RL method: the model tries an answer several times; better-than-average tries get reinforced. Sharpens reasoning.

Contents

One-page cheat-sheet

The four generations (the arc)

- **Cosmos 1 — Toolbox:** two engines (diffusion + autoregressive), T5 text, separate tasks.
- **Cosmos 2 — Polish:** diffusion-only + sparse attention (up to 2.6× faster).
- **Cosmos 2.5 — Unify:** one flow model, Reason1 encoder, RL, 30s multi-camera.
- **Cosmos 3 — Leap:** two-tower mixture-of-transformers omnimodel; +audio +action.

The platform: families (not one model)

Family	Job	Technique
Predict	generate the future world	diffusion + autoregressive (1) → flow (2.5)
Transfer	controllable sim→real	diffusion DiT + ControlNet
Reason	understand & decide	autoregressive-text VLM (+ SFT/RL)
Tokenizer	compress video ↔ tokens	autoencoder (continuous + discrete)

Jargon in one line each

- **Diffusion** = sculptor: refine noise → video over many steps (top quality, slower).
- **Autoregressive** = writer: predict the next token (fast; over video tokens in Predict, over text tokens in Reason).
- **Flow matching** = a near-straight noise→data path → far fewer steps.
- **Sparse attention (NATTEN)** = each patch attends only to nearby patches → up to 2.6× faster.
- **Mixture-of-Transformers** = each modality/tower has its own weights but shares global attention.
- **SFT** = teach with curated Q→A; **GRPO** = RL that rewards better-than-average tries.

Numbers worth remembering

- **Predict 1:** diffusion 7B/14B, autoregressive 4B/12B; ~100M clips from 20M hours; ~10k H100s.
- **Predict 2.5:** 2B/14B; 200M clips; PAI-Bench 0.810; up to 30s, multi-camera.
- **Reason:** 1 = 7B/56B, 16K context; 2 = 2B/8B, 256K context, OCR + 2D/3D.
- **Cosmos 3:** Nano 16B (8B+8B), Super 64B (32B+32B); ~20T training tokens (press-reported).

One sentence: Cosmos went from "many tools that dream video" → "the best tool, polished" → "one smart unified tool" → "a reasoning, seeing, hearing, acting omnimodel."

The big picture: all four generations side by side

	Cosmos 1	Cosmos 2	Cosmos 2.5	Cosmos 3
Released	Jan 2025	Jun 2025	Sep 2025	Jun 2026
Core engine	TWO engines: diffusion + GPT-style	Refined diffusion	ONE unified flow-based model	Mixture-of-Transformers (reason + generate)
Inputs	Text, or image/video	Text, or image/video	Text/image/video (one model)	Text, image, video, AUDIO + ACTIONS
Outputs	Future video	Video (480/704p)	Video up to 30s, multi-camera	Video + actions + audio, with reasoning
Reads text via	T5-XXL encoder	T5-style encoder	Cosmos-Reason1 (VLM)	Built-in reasoning transformer
Model sizes	Diff 7B/14B, GPT 4B/12B	0.6B / 2B / 14B	2B / 14B (+auto/robot variants)	Super / Nano / Edge
Training data	~100M clips (from 20M hrs)	+ action datasets	200M curated clips	~20 trillion tokens
Post-training	Video2World fine-tune	Action-conditioned fine-tune	Model merge + RL	SFT + RL (GRPO-style)
One-word vibe	Kitchen sink	Polish	Unify	Leap

Table 1. Each column is colour-coded and reused throughout this guide: Cosmos 1, Cosmos 2, Cosmos 2.5, Cosmos 3.

Important: Cosmos is a PLATFORM, not a single model

So far we compared “generations” (1, 2, 2.5, 3). But each generation is really a **bundle of separate model families that do different jobs**. The autoregressive 4B model, for example, is just one variant inside the **Predict** family of generation 1. There are three core “pillars” plus shared tools.

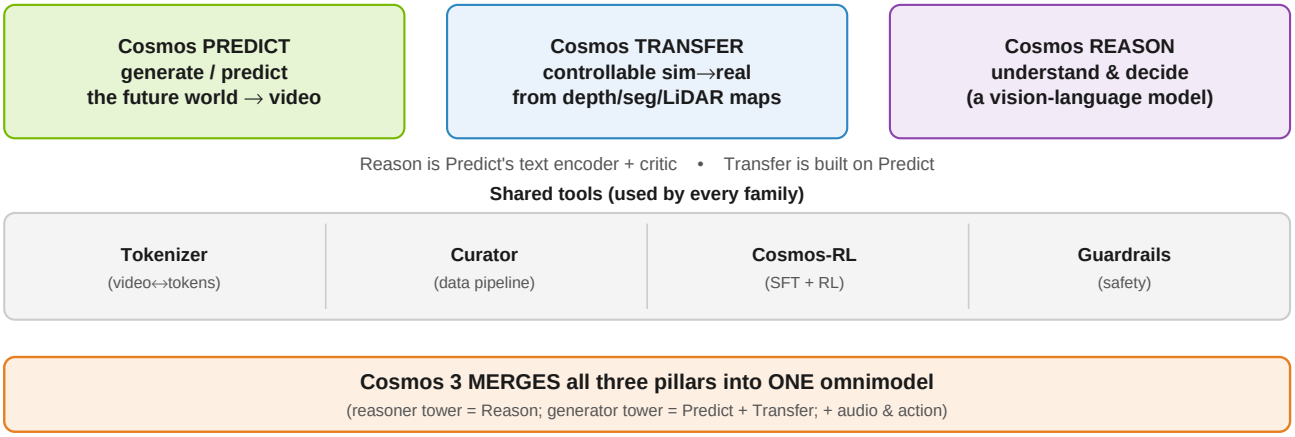


Figure 1. Three pillars (Predict / Transfer / Reason) sit on shared tools. They interconnect — and by Cosmos 3 they fuse into a single model.

The model families at a glance

Family	Its job	Input → Output	Technique	Versions
Predict	Generate / predict the future world	text/image/video → video	diffusion + AR (v1) → diffusion (v2) → flow (v2.5)	1, 2, 2.5
Transfer	Controllable generation; sim→real	structure maps (depth, seg, edge, LiDAR, HD-map) + text → photorealistic video	(Multi-)ControlNet on top of Predict	1, 2.5
Reason	Understand & decide	image/video + text → text reasoning, decisions, (v2) 2D/3D detections	vision-language model (VLM)	1, 2
Tokenizer	Compress video ↔ tokens	video → continuous/discrete tokens	causal autoencoder (FSQ)	1
Curator	Prepare training data	raw video → clean, captioned dataset	Ray GPU pipeline	—
Cosmos-RL	Train / fine-tune any family	checkpoints + data → fine-tuned models	SFT + RL	—
Guardrails	Safety	prompts/outputs → filtered content	classifiers	—

Table 2. Predict, Transfer and Reason are different MODELS for different jobs — not versions of one another. Generations 1/2/2.5 ship them separately; Cosmos 3 unifies them.

Diversity inside ONE family: Cosmos Predict 1

To show how much sits inside a single family-and-generation: **Cosmos Predict 1 alone** is about a dozen distinct checkpoints. The “autoregressive 4B” described earlier is just one of them.

Variant	Type	What it does
Diffusion Text2World 7B / 14B	diffusion	text → video
Diffusion Video2World 7B / 14B	diffusion	image/video → future video
Autoregressive 4B / 12B	GPT-style	video → next frames (the one described earlier)
Autoregressive 5B / 13B Video2World	GPT-style	text + video → future video
Tokenizer CV / DV (×3 rates each)	tokenizer	compress / decompress video
Prompt Upsampler 12B	LLM	expand short prompts into rich captions
Diffusion Decoder 7B	diffusion	clean up the autoregressive output

Table 3. One family, one generation ≈ a dozen checkpoints. Multiply by Transfer + Reason + tools to see why “Cosmos 1” is a platform, not a model.

How the families work together

- **Reason → Predict:** Reason1 is reused as the **text encoder** inside Predict 2.5, and as a **quality critic** during data curation.
- **Predict → Transfer:** Transfer 2.5 is **built on top of** Predict 2.5 — a ControlNet wrapped around it so you can steer generation with depth/segmentation/LiDAR maps.
- **Transfer's special role:** it doesn't invent freely; it **converts a structured input** (e.g. a simulator's segmentation video) into photorealistic video — the **sim-to-real** and data-augmentation workhorse.
- **Tokenizer / Curator / Cosmos-RL / Guardrails** feed and support every family.

Takeaway: don't read Cosmos as one model getting bigger. Read it as a platform of specialised families (Predict = imagine, Transfer = restyle/control, Reason = think) that generation 3 finally fuses into a single omnimodel.

Each family evolved on its OWN cadence (not in lockstep)

A subtle but important point: Predict, Transfer and Reason did **not** all version together. Predict went 1→2→2.5; Transfer jumped 1→2.5 (no public "2"); Reason has its own 1→2 track. The "generation" labels really follow **Predict**. And each generation is **not** equally "diverse":

Era	Predict	Transfer	Reason	How diverse?
Cosmos 1 (2025 H1)	Predict 1	Transfer 1	Reason 1	Full platform
Cosmos 2 (mid 2025)	Predict 2	—	—	Mostly a Predict-only refresh
Cosmos 2.5 (Sep 2025)	Predict 2.5	Transfer 2.5	Reason 1 reused (Reason 2 later)	Full platform again
Cosmos 3 (Jun 2026)	→ generator tower	→ folded in	→ reasoner tower	Unified into ONE model

Table 4. Cosmos 2 was a Predict upgrade; 2.5 restored the full platform; Cosmos 3's novelty is that it STOPS being a platform of separate models and fuses them.

The three families converging into Cosmos 3

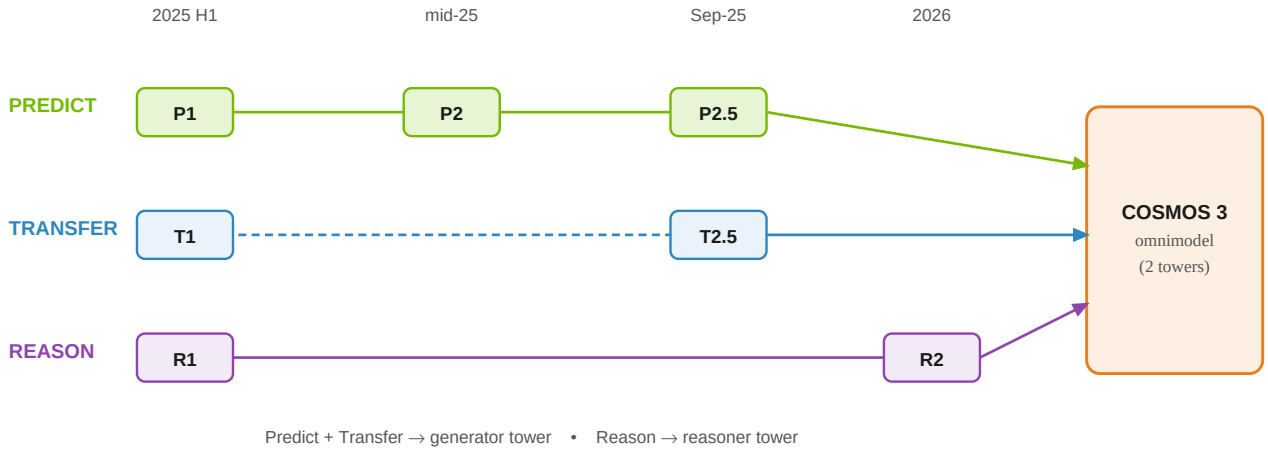


Figure 2. Dashed Transfer lane = it skipped a public "2". All three families converge into Cosmos 3's two-tower omnimodel.

Same family, across versions: the novelty at each step

Family	Evolution (version → novelty)	Fate in Cosmos 3
Predict	1: two engines (diffusion 7/14B + autoregressive 4/12B), T5 encoder, separate tasks. 2: diffusion-only DiT (2/14B) + sparse attention → up to 2.6× faster. 2.5: flow-based + unified Text/Image/Video-to-World + Reason1 encoder + RL + 30s multiview.	Becomes the generator tower
Transfer	1: Multi-ControlNet on Predict 1; controls = depth/edge/keypoint/segmentation/LiDAR/HD-map; 4 control blocks at the START of the network; 7B. 2.5: built on Predict 2.5; control blocks DISTRIBUTED (one every 7 blocks); 2B is 3.5× smaller than Transfer1-7B with better alignment & less hallucination.	Folded into the generator tower (conditioning / control)
Reason	1: physical-AI reasoning VLM (7B/56B), NVIDIA hybrid Mamba-MLP-Transformer backbone, 16K context, SFT + RL (GRPO), long chain-of-thought. 2: NEW backbone (post-trained from Qwen3-VL), 2B/8B/32B, 256K context (16×), adds OCR + 2D/3D point localization + bounding boxes + timestamp precision; #1 open model.	Becomes the reasoner tower
Tokenizer	1: causal autoencoder; continuous (CV) for diffusion + discrete (DV, FSQ 64k vocab) for autoregressive; up to 12× faster than prior tokenizers.	Reused as the visual/audio encoder family (ViT/VAE)

Table 5. Read each row left→right to see one family's novelty trajectory, and the right column for where it ends up inside Cosmos 3.

The one-paragraph synthesis

Cosmos 1 and 2.5 are **full platforms** (Predict + Transfer + Reason + Tokenizer); Cosmos 2 was mainly a **Predict** efficiency/quality upgrade (sparse attention); Reason ran its own **1→2** track (long context + spatial grounding). The big architectural story of **Cosmos 3** is not a new family — it is the **disappearance** of separate families: the generator tower absorbs Predict + Transfer, the reasoner tower absorbs Reason, and audio + action are added on top.

How the architecture changed (diagrams)

Cosmos 1 — two parallel engines

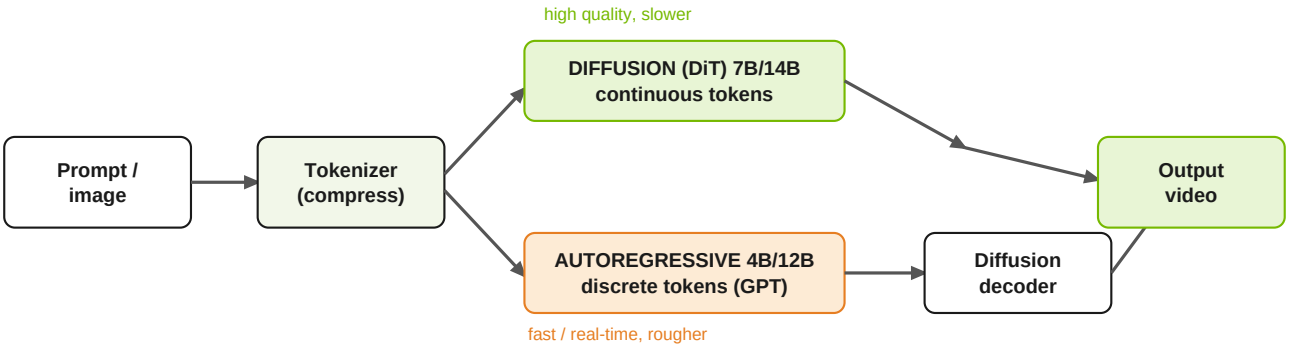


Figure 3. Cosmos 1 shipped two separate engines. Diffusion (top) = best quality; autoregressive (bottom) = fast, but needs a diffusion decoder to clean up its output. Text enters both via a T5-XXL encoder (not shown).

Cosmos 2.5 — one unified flow model with a “smart” text encoder

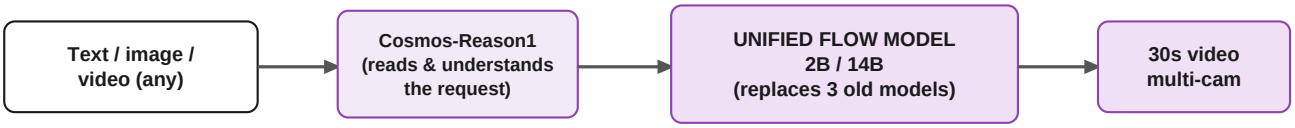


Figure 4. Cosmos 2.5 merged the three separate task-models of earlier versions into a single flow-based model, and upgraded the text encoder to Cosmos-Reason1 — a model that actually understands physics and language.

Cosmos 3 — mixture-of-transformers (thinks before it dreams)

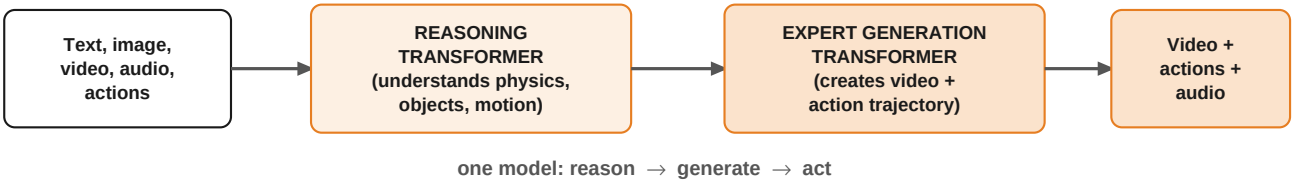


Figure 5. Cosmos 3 is no longer just a video predictor. A reasoning transformer first understands the scene, then an expert generation transformer produces video AND physical actions (and audio) — all in one “omnimodel”.

Deep-dive: why did Cosmos 1 have TWO engines?

This is the most confusing part of Cosmos 1, so it deserves a clear answer. The two engines were **not** a duplicate or a mistake — the paper states it built them on purpose: *"We explore two scalable approaches for building pre-trained world foundation models — the diffusion model and the autoregressive model."* Both solve the **same** hard problem (predict believable future video) with the same trick — break one hard generation into many easy sub-steps — but in two different "spaces", which forces two different training recipes.

> *Analogy: same destination, two routes. Diffusion is a **sculptor** — start from a rough noisy block and refine the whole video over many passes. Autoregressive is a **writer** — produce the video one "token" at a time, left to right, like an LLM.*

Aspect	Diffusion engine (sculptor)	Autoregressive engine (writer)
Token space	Continuous latents (CV 8×8×8 tokenizer)	Discrete codes (DV 8×16×16; 64,000-word visual vocabulary)
Learning signal	Denoising score-matching (EDM): recover the clean signal from a noised one	Next-token cross-entropy: predict the next code (exactly like an LLM)
Text conditioning	T5-XXL via cross-attention	T5 via cross-attention added to blocks
Training stages	Text2World pretrain → Video2World fine-tune (add observed frames)	Video-only next-token pretrain → add text for Video2World
Model sizes	7B / 14B	4B / 12B (5B / 13B for Video2World)
Strength	Highest visual fidelity	Speed + streaming; plugs into the LLM toolbox (KV-cache, real-time)
Weakness	Many denoising steps → slower	Heavy compression → artifacts → needs a diffusion decoder to repaint detail

Table 6. The two engines use different tokenizers, different losses, and different training stages — they are genuinely different models, not one model trained twice.

So why keep both?

They are **complementary bets**. Diffusion is the **quality** champion; autoregressive is the **speed + LLM-integration** champion (it treats video like language — exactly the direction Cosmos 3 later embraced). The **diffusion decoder** exists only because the autoregressive path's aggressive discrete compression *"can sometimes lead to undesired distortions"* — so a small diffusion model cleans its output back up.

Important: does this two-engine split apply to the OTHER families?

No. The diffusion-vs-autoregressive choice is a **Predict** thing. Transfer, Reason and the Tokenizer each use the one technique that fits their job:

Family (gen 1)	Diffusion?	Autoregressive?	Core technique
Predict 1	Yes (7B/14B)	Yes (4B/12B)	BOTH — the only two-engine family
Transfer 1	Yes	No	diffusion DiT + ControlNet (wraps Predict's diffusion)
Reason 1	No	Yes — over TEXT tokens	decoder-only VLM (+ SFT/RL); "distinct from diffusion"
Tokenizer 1	No	No	autoencoder — outputs continuous (→diffusion) & discrete (→AR) tokens

Table 7. Only Predict has the two-engine split. Note "autoregressive" means different things: Predict predicts the next discrete VIDEO token (future frames); Reason is a normal LLM/VLM predicting the next TEXT token. In Cosmos 3 these become the two towers — generator (diffusion, Predict+Transfer) and reasoner (autoregressive-text, Reason).

Keynote: the ONE factor that separates the two engines

Both engines are really **conditional video generators** that learned spatial + temporal physics from video (the “world model” label is about the *goal*, not the mechanism). They differ on a **single fundamental axis** — and almost everything else follows from it: **how, and in what order, they generate.**

DIFFUSION — refine the WHOLE clip at once (continuous, bidirectional)

AUTOREGRESSIVE — predict the NEXT piece from the past (discrete, causal)

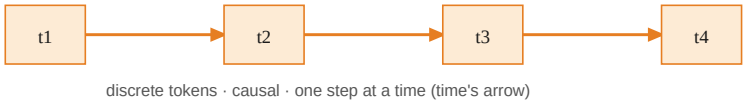


Figure 6. The deep difference: diffusion improves the entire clip together over many steps (it can see past AND future); autoregressive builds the future one step at a time from the past only — like time itself.

Everything else cascades from that one axis

	Diffusion (holistic)	Autoregressive (causal)
Representation	continuous 16-dim latents	discrete 64k codes (lossy)
Objective	denoising score-matching	next-token cross-entropy
Context	bidirectional (whole spacetime)	causal (past only)
Clip length	fixed (set up front)	arbitrary / streaming / extendable
Error over time	globally consistent, no drift	accumulates drift
Speed / quality	many steps → slower, top fidelity	KV-cache → real-time; lossy (needs decoder)
Lineage	vision / graphics	language models

Table 8. Each row is a downstream consequence of the single “generation order + representation” difference at the top.

Why it matters for a WORLD model

The physical world is **causal** — the future follows from the past, and a robot acts **step by step**. Autoregressive next-token prediction **mirrors time's arrow**, which is why it is the natural path to **interactive, streaming, real-time** world simulation. Diffusion treats a clip as a block to be perfected holistically, which is why it wins on **per-clip realism and global coherence**. That single trade-off is exactly why Cosmos kept both — and why **Cosmos 3 uses each for what it is best at**: the autoregressive **reasoner** tower (causal understanding) + the diffusion **generator** tower (high-fidelity creation).

Easy explainer: “bidirectional” vs “causal next-token”

These two words are the heart of the difference, so here they are in plain language. It all comes down to one thing: **which other tokens is a token allowed to “look at”?** (A token = a small chunk of the video — a patch, or a moment in time; “attention” = each token looking at others to decide what it should be.)

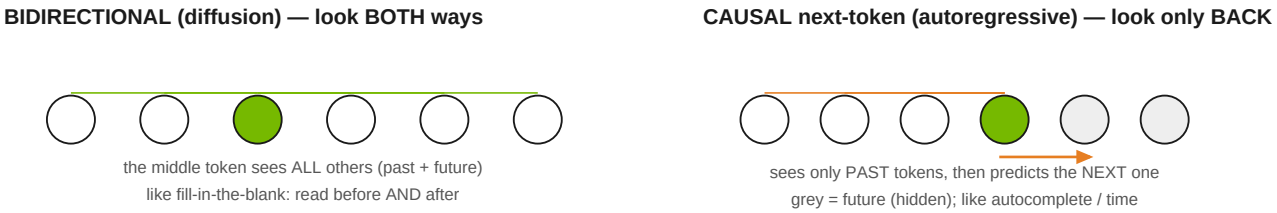


Figure 7. Left: in diffusion the whole clip exists at once, so a token attends to every other token (before and after). Right: autoregressive builds left-to-right, so a token may attend only to the past and then predicts the next token; the future is hidden.

Bidirectional (diffusion) — “look both ways”

When diffusion generates, the **whole clip is present from the start** (just noisy at first), so every token can look **both directions** — before AND after it. It is like a **fill-in-the-blank**: to guess a missing word you read the words on both sides. Because all tokens are refined together, each one always has the **full surrounding context** — which gives strong **global consistency** (the start and end of the clip “agree” because they saw each other).

Causal next-token (autoregressive) — “look back, guess the next”

Autoregressive generates **one token at a time, in order**, like typing. When predicting token #5, tokens #6, #7… **don't exist yet**, so it can only look **backward** and guess #5, then append it and guess #6, and so on. **“Causal”** = cause-before-effect: the future is built **from** the past, never peeking ahead — exactly like **time**. (In training a **causal mask** hides the future; diffusion uses no mask — full attention.)

The trade-off in everyday terms

- **Bidirectional** sees everything → great coherence, but must produce the **whole fixed-length block at once**; there is no natural “next”, so it can't easily stream or continue.
- **Causal** sees only the past → it can **keep going forever** (stream / extend), matches real time and step-by-step robot action, and is fast with caching — but since each guess can't use future context, small mistakes can **snowball (“drift”)** over long videos.

One line: bidirectional = “fill in the whole picture using all surrounding context at once”; causal next-token = “write the future one step at a time, using only what already happened.”

Why each new generation? (the problem it solved)

A fair question: was each new version just a **bigger model on more data**? **No.** Scale grew every time, but it was the *enabler*, not the headline — each step introduced a distinct architectural or algorithmic idea to fix a concrete problem.

Step	Key problem with the previous version	How they solved it	Real novelty (beyond size/data)
1 → 2	Too slow, too many separate engines, visible hallucinations — a research platform, not a daily tool.	Commit to the diffusion path; add sparse attention; engineer better quality/control; add small fast variants; add action-conditioned post-training.	Sparse attention (≈2.6× faster) + action conditioning.
2 → 2.5	Three separate task-models to juggle; a T5 text encoder that doesn't "understand" physics; short, single-camera video.	Unify the 3 tasks into ONE flow-based model; replace T5 with the Cosmos-Reason1 VLM as encoder; add RL post-training + model merging; extend to 30s, multi-camera.	Task unification + flow matching + reasoning-VLM encoder + RL.
2.5 → 3	Still only a video predictor — can't natively reason, hear, or act; understanding and generation were separate models.	One mixture-of-transformers omnimodel: a reasoning transformer + an expert generation transformer; reasons before it generates; adds audio + action.	New architecture (MoT) + new modalities (audio/action) + merging understand/generate/act.

Table 9. Read the rightmost column: every jump is a genuine idea, not a scale-up. 1→2 buys speed; 2→2.5 buys understanding + simplicity; 2.5→3 buys a whole new capability.

The one-sentence "why"

- **1→2:** "Make it fast and reliable enough to actually use."
- **2→2.5:** "Make it one model that truly understands the request."
- **2.5→3:** "Stop just watching the world — reason about it, hear it, and act in it."

The data: scale and diversity

A WFM is only as good as the video it learns from. Cosmos 1 curated **~100 million clips** out of **20 million hours** of raw video, deliberately balanced across nine kinds of physical scene so the model isn't biased toward one (e.g. only driving). Later generations grew the data by orders of magnitude.

Cosmos 1 training-data mix (by category)

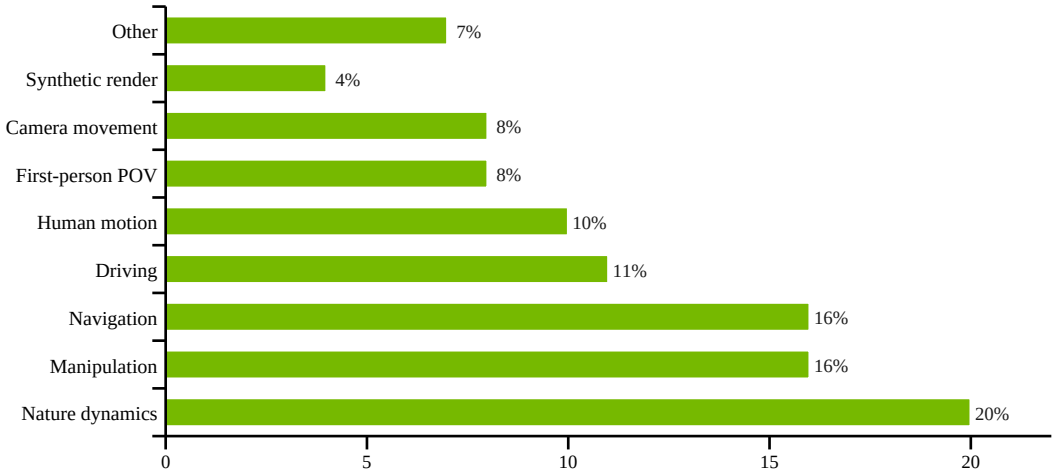


Figure 8. Diversity is intentional: nature, robot manipulation, and navigation dominate, but driving, human motion and synthetic data are all represented. This breadth is what lets one pretrained model be fine-tuned for many downstream robots/vehicles.

Data scale across generations

Generation	Pretraining data	Notes
Cosmos 1	~100M clips (from ~20M hours)	9 balanced categories; ~30% removed by dedup
Cosmos 2	Cosmos 1 data + action datasets	Bridge, AgiBotWorld, GR00T Dreams for post-training
Cosmos 2.5	200M curated high-quality clips	+ domain post-training data (auto, robotics)
Cosmos 3	~20 trillion tokens	~1B images, ~400M videos, + audio + action trajectories

Table 10. The jump from “100M clips” to “20T tokens” reflects Cosmos 3 also ingesting text, audio and robot/human action data — not just video. **Caveat:** Cosmos 3's exact counts (20T tokens, ~1B images, ~400M videos) come from press coverage; NVIDIA's official release states only “billions of samples across text, image, video, sound and action.”

The key insight: each generation adds a new TYPE of data

The data story is **qualitative**, not just “more”. Each generation introduces a new *kind* of data, and that new kind is exactly what unlocks the next capability:

- **Cosmos 1 — broad video** (9 balanced categories) → learns **general physics**.
- **Cosmos 2 — + action-labelled robot data** (Bridge, AgiBotWorld, GR00T Dreams) → learns **“this action causes that outcome.”**
- **Cosmos 2.5 — + multi-camera / domain data** (7-cam driving, 3-cam robotics) → learns **deployment realism and multi-view consistency.**
- **Cosmos 3 — + audio and action trajectories** (from humans and robots) → enables the **omnimodel** that can hear and act, not just see.

Model sizes & key hyperparameters

“Parameters” (B = billion) are the model's adjustable knobs — more usually means smarter but slower/heavier. Cosmos offers small *and* large versions so you can trade quality for speed.

Every model variant at a glance

Generation / family	Key models (sizes)	Helper / notes
Cosmos 1	Predict: Diffusion 7B & 14B + Autoregressive 4B & 12B (5B/13B Video2World) · Reason 1: 7B / 56B	Tokenizer 77–105M; Prompt-upsampler 12B; Diffusion-decoder 7B
Cosmos 2 (Predict 2)	Text2Image 0.6B / 2B / 14B · Video2World 2B / 14B (+ 2B action-conditioned, 14B GR00T-Dreams samples)	Sparse attention (NATTEN), up to 2.6×
Cosmos 2.5	Predict 2.5: 2B & 14B (+ auto 7-cam, robot 3-cam) · Transfer 2.5: 2B	Cosmos-Reason1 as text encoder
Reason 2 (2026)	2B / 8B / 32B (post-trained from Qwen3-VL)	256K context; OCR + 2D/3D localization; #1 open
Cosmos 3	Nano 16B (8B+8B) · Super 64B (32B+32B) · Edge (soon)	Two-tower mixture-of-transformers

Cosmos 1 diffusion hyperparameters (the most fully documented)

Setting	7B model	14B model
Transformer layers	28	36
Model (hidden) dimension	4,096	5,120
Attention heads	32	40
AdaLN-LoRA rank	256	256
Learning rate	2^-15	2^-16
Optimizer	AdamW (beta1=0.9, beta2=0.99), weight decay 0.1–0.2	
Warmup	2,500 iterations, linear	
Text encoder	T5-XXL, 512 tokens via cross-attention	
Resolution schedule	512p (57 frames) → 720p (121 frames)	
Context length @720p	56,320 tokens	
Diffusion formulation	EDM denoising score-matching; QK-RMSNorm for stability	
Compute	~10,000 H100 GPUs, ~3 months	

Table 11. NVIDIA published far more hyperparameters for Cosmos 1 than for the closed-er later releases; 2.5 and 3 reuse the same design DNA (3D-RoPE positions, AdaLN-LoRA, a causal tokenizer) at larger scale.

Inputs and outputs: the “data contract” (DTO)

Think of a **DTO** (data-transfer object) as the exact shape of what you hand the model and what it hands back. This shape widened a lot across generations.

Generation	INPUT (what you provide)	OUTPUT (what you get back)
Cosmos 1 (diffusion)	Text → T5-XXL embedding (512 tokens) + optional conditioning frames + fps + frame-count	Continuous latent video → decoded to RGB frames
Cosmos 1 (autoregressive)	Discrete video tokens (DV 8×16×16) + T5 text embedding	Next discrete tokens → diffusion-decoder → RGB frames
Cosmos 2	Text, or image, or video + fps	RGB video at 480p / 704p, 10–16 fps
Cosmos 2.5	Text / image / video, encoded by Cosmos-Reason1	Up to 30s video; optional multi-camera (3–7 synced views)
Cosmos 3	Text + image + video + ambient audio + action trajectories	Video + action trajectory + audio, preceded by explicit reasoning

Table 12. The input distribution broadens from “text + frames” (Cosmos 1) to a full five-modality stream including audio and actions (Cosmos 3). The output likewise grows from “a short video” to “video + the physical actions a robot should take”.

Why the input distribution matters

- **Cosmos 1–2:** trained mostly on curated *internet-style* video across 9 categories — great general physics, but you fine-tune for your specific robot.
- **Cosmos 2.5:** 200M *physical-AI-focused* clips + domain post-training (driving, manipulation) shift the distribution toward real deployment.
- **Cosmos 3:** adds *action trajectories from real humans and robots* and *audio* to the training mix — so the input/output distribution now includes the robot's own behaviour, not just what a camera sees.

Key results: inference speed

Speed is the headline practical difference. Faster generation = more simulations per day = faster robot/AV development. Each generation attacked latency differently.

Generation	Speed lever	Reported result
Cosmos 1	Causal, factorized tokenizer	Up to 12× faster encode/decode vs. prior tokenizer (CogVideoX)
Cosmos 2	Sparse attention in the DiT	Up to 2.6× end-to-end inference speedup
Cosmos 2.5	Flow-based sampling (fewer steps) + smaller 2B option	Faster, more resource-efficient than 2.x diffusion
Cosmos 3	Dedicated Nano variant	High-quality video + action reasoning in a fraction of a second

Table 13. Two concrete, comparable speedups are published: Cosmos 1's tokenizer (up to 12×) and Cosmos 2's sparse attention (up to 2.6×). 2.5 and 3 emphasise efficiency via flow sampling and a sub-second Nano tier rather than a single headline multiplier.

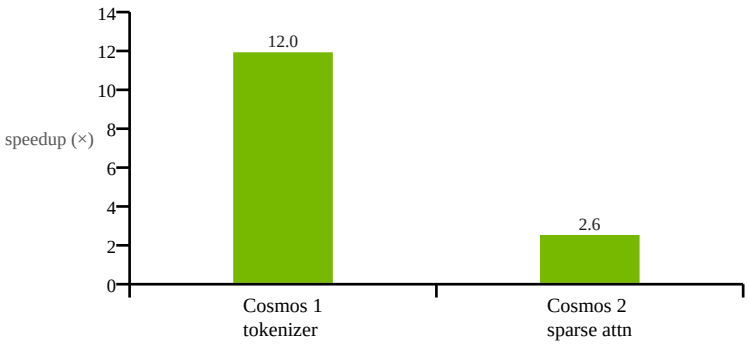


Figure 9. The two directly-quoted "× speedup" numbers. (Different things are being sped up — tokenizer vs. whole pipeline — so this compares *magnitude of improvement*, not absolute latency.)

The quality/speed trade-off in one sentence

Diffusion (Cosmos 1–2) = many denoising steps = top quality but slower; **autoregressive** (Cosmos 1) = stream tokens = fast but rougher; **flow-based** (Cosmos 2.5) = fewer steps for similar quality; **Nano** (Cosmos 3) = engineered for sub-second responses.

Zoom-in: what is “sparse attention”? (the Cosmos 2 trick)

Cosmos 2's headline speedup (up to 2.6×) comes from **sparse attention**. Here is the idea with no maths.

The problem: “attention” is very expensive for video

Inside a Transformer, every token (think: every little patch of the video) **looks at every other token** to decide what is relevant. That is called **full attention**. A short 720p video is **tens of thousands of patches** (frames × height × width), and full attention compares *every patch with every other patch* — so the cost grows **quadratically**: double the patches → **4×** the work. For video, this one step dominates the whole generation time.

The insight: most of those comparisons are wasted

A patch in the top-left of frame 1 has almost nothing to do with a patch in the bottom-right three seconds later. What actually matters to a patch is its **neighbours** — nearby in space, and a frame or two before/after.

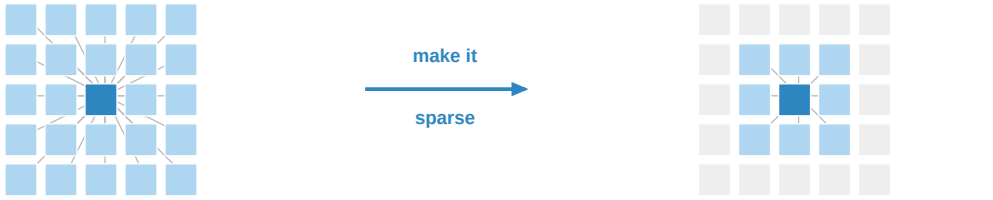


Figure 10. Left: every patch attends to all others (lines everywhere). Right: each patch attends only to its local window — the grey patches are skipped. Cosmos 2 (via the NATTEN library) drops up to 98% of the connections, keeping only the ~2% that matter.

The fix: Neighbourhood Attention (NATTEN)

Instead of attending to **all** patches, each patch attends only to those in a small **local window** around it (in space and time). Cosmos 2 pushed this so **up to 98% of the attention connections are dropped** (sparsity raised “from 50% to 98%”) — only the most relevant ones are computed.

- **Result:** far fewer calculations → **1.7×–2.6× faster** at 720p, with quality essentially preserved (the dropped links were the unimportant ones).
- **Bonus:** it is an inference-time efficiency change, not a bigger model — related “Generalized Neighbourhood Attention” can even be plugged into existing models for 28–46% speedups **without any retraining**.

Analogy: full attention = everyone in a stadium trying to talk to everyone else at once (chaos, won't scale). Neighbourhood attention = each person only talks to their own row and the rows just ahead/behind — you keep all the context that matters, and it scales.

Zoom-in: what is “flow matching”? (the Cosmos 2.5 trick)

Cosmos 2.5 switched from **diffusion** to a **flow-based** model. Here is what that means, again with no maths.

Diffusion takes a wandering path

Recall the diffusion “sculptor”: it turns noise into video with **many tiny denoising steps**. The path it follows from pure noise to a finished video is **curved and wandering**, so it needs *lots* of small steps (often 30–50+) to stay on track. Each step is a full pass through a huge model — so many steps = slow.

Flow matching learns a straight “current” instead

Picture noise on the left and real video on the right. Flow matching teaches the model a **direction to move** at every point — “which way, and how fast, to get from noise toward data”. It trains by connecting each noise sample to a real sample with a **straight line** and learning to follow it. Because the target paths are **nearly straight**, you can take **big steps** and still land in the right place — so you need **far fewer steps**.

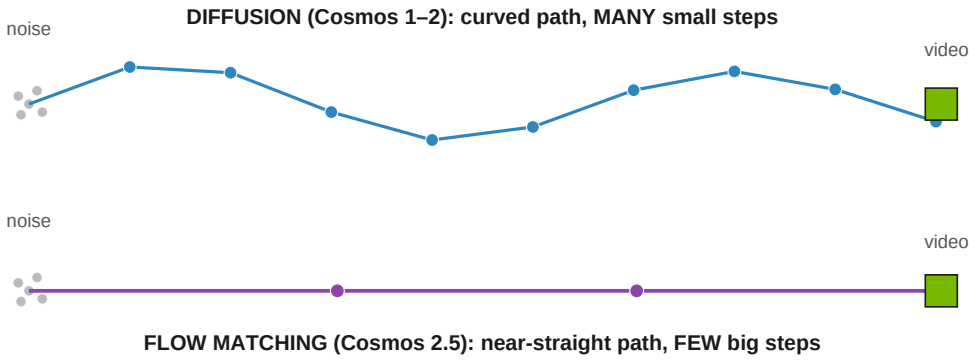


Figure 11. Same start (noise) and destination (video). Diffusion zig-zags there in many small steps; flow matching learns a near-straight route it can cover in a few big steps.

Why Cosmos 2.5 switched to it

- **Fewer sampling steps** → **faster & more resource-efficient** — important for 30-second, multi-camera video.
- **A cleaner, more stable training objective** (simply “match the direction”), which made it easier to build **one unified model** for Text/Image/Video-to-World instead of three.

Analogy: diffusion is a windy mountain road with dozens of turns (many careful steps); flow matching is a straight highway to the same town — fewer, bigger moves to arrive.

Deep-dive: how Cosmos 3 fuses 5 modalities (incl. audio)

Cosmos 3 must ingest AND produce **five** very different data types — text, image, video, **audio**, **action** — and reason about them together (text is symbols, video is pixels, audio is a waveform, action is control vectors). How do you put all of that into **one** model? Three ingredients:

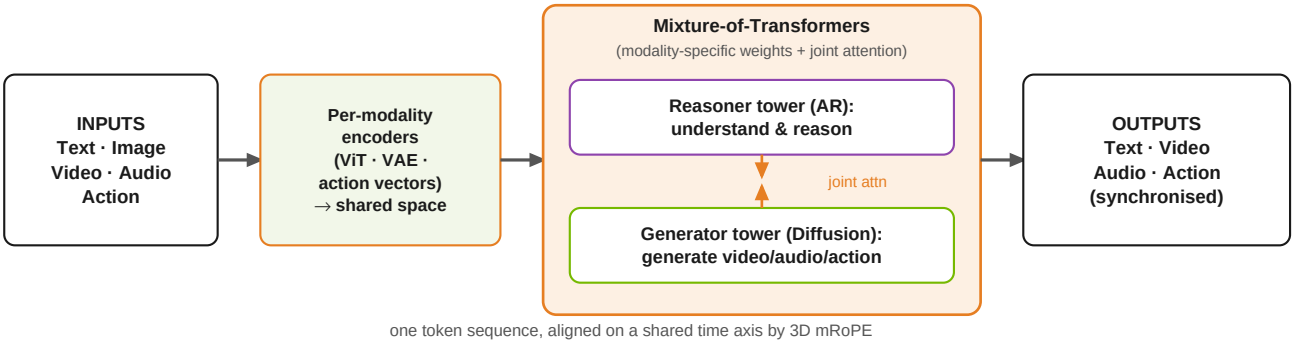


Figure 12. Each modality is encoded into a shared space, concatenated into one sequence, and processed by a two-tower Mixture-of-Transformers: separate weights per modality/tower, but one global attention so audio, video, text and action all "see" each other.

1) A dedicated encoder per modality → one shared space

Modality	How it enters the shared space
Text	Token embeddings (like an LLM)
Image / Video	A ViT for understanding; a VAE for generation
Audio	A VAE encodes the sound, then a linear projection maps audio tokens into the hidden dimension
Action	Domain-aware action vectors (robot / AV control)

2) One sequence + global attention, aligned by 3D mRoPE

All tokens are concatenated into **one sequence** with **global self-attention** — an audio token can attend to a video token can attend to an action token. A unified **3D mRoPE** puts video, audio and action tokens **on one shared temporal axis** — literally how the model knows which sound goes with which frame and which action.

3) Mixture-of-Transformers — the "one group" mechanism

In a plain transformer all modalities fight over the same weights. **MoT gives each modality/tower its own weights** (feed-forward, attention projections, layer-norms) **while sharing global attention** — specialisation without fragmentation. The **Reasoner** (AR = understand) and **Generator** (diffusion = create) towers use separate parameters but **joint attention**; the reasoner can run alone, generation activates both. Sizes: **Nano 16B** (8B+8B), **Super 64B** (32B+32B).

Managing the audio dataset specifically

- Audio is **ambient sound paired with video**, kept **temporally synchronised** and VAE-encoded; aligned to video/action via the shared mRoPE time axis.
- Curated through the same **multi-stage pipeline** (filter + quality review) as the rest of the data, from NVIDIA-owned + commercially-permissive sources.
- NVIDIA explicitly lists **"inaccurate sound–video alignment"** as a known failure mode — audio↔video timing is the central hard problem (and why mRoPE matters). *Granular audio-dataset counts are not publicly disclosed.*

Does Cosmos 3 reason THEN predict, or do it all at once?

A common question: are reasoning and generation two separate steps? **No — they happen jointly in one forward pass.** This is the key difference from the old platform, where Reason and Predict were literally **two separate models run in sequence**.

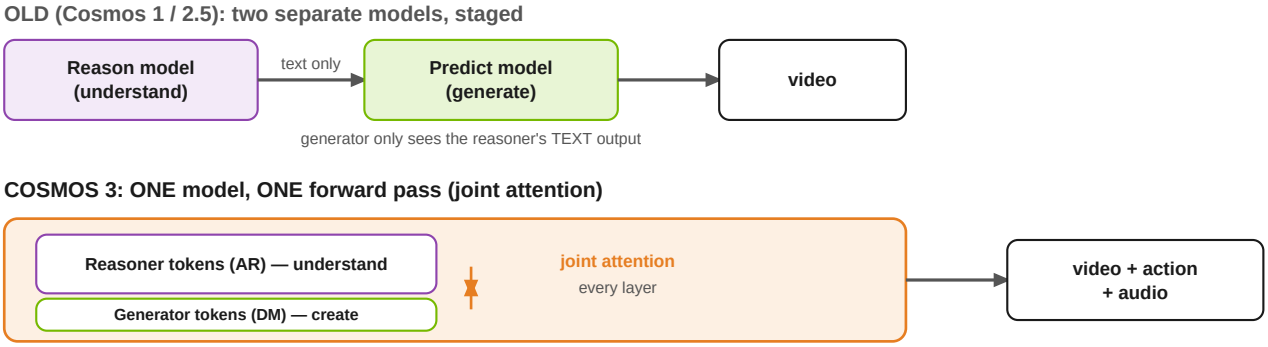


Figure 13. Old = a pipeline of two models; the generator only received the reasoner's text. Cosmos 3 = one model where reasoning and generation tokens share attention at every layer.

The precise picture

- **One unified forward pass:** NVIDIA states Cosmos 3 “can reason and generate … in one unified forward pass” — not a two-model pipeline.
- **Separate weights, joint attention:** the two towers have their own parameters (the Mixture-of-Transformers trick) but “interact through joint attention” at every layer — so they are **coupled, not deferred** to separate stages.
- **“Reasons before generating” is an INFORMATION order, not a separate model call:** the reasoning tokens sit earlier in the sequence and the generation tokens attend back to them, so reasoning **conditions** generation within the same pass. (Typically causal attention over the reasoning tokens + bidirectional within the diffusion block — the standard pattern for hybrid AR+diffusion models; NVIDIA hasn't published the exact masking for Cosmos 3.)
- **The one staged case:** the reasoner **can run alone** as a pure VLM (understanding only). But whenever it generates, **both towers run together**.

Why joint beats staged: in the old pipeline the generator only saw the reasoner's text summary; in Cosmos 3 it can attend to the reasoner's **full internal representations**, and reasoning can be informed by what is being generated — tighter coupling means fewer hallucinations, better physical consistency, and one efficient pass instead of two model invocations.

Pretraining and post-training: SFT and GRPO

Modern Cosmos models are built in two phases. **Pretraining** soaks up broad knowledge from massive data. **Post-training** then sharpens behaviour with curated examples (**SFT**) and reinforcement learning (**GRPO**). This recipe is clearest in **Cosmos-Reason1** — the reasoning brain that powers Cosmos 2.5's text understanding and the spirit of Cosmos 3.

The 4-stage training pipeline (Cosmos-Reason1)

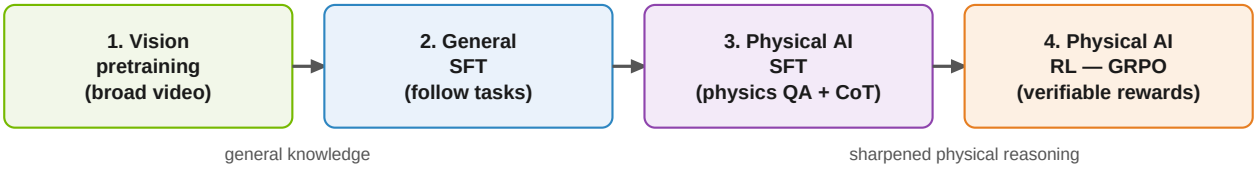


Figure 14. Knowledge first (stages 1–2), then physical-world specialisation (stage 3), then reinforcement learning to make the reasoning reliable (stage 4).

What happens in SFT (stage 3)

- Curated **question** → **answer** examples for physical common sense (space, time, basic physics) and embodied decisions (what should the robot do?).
- Built from real robotics/AV datasets: **BridgeData V2, RoboVQA, AgiBot, HoloAssist**, and driving data.
- The team hand-picked **~200–250 examples per data source**, turned them into multiple-choice questions, and removed ambiguous ones — quality over quantity.
- Answers include **chain-of-thought** (the model explains its reasoning step by step).

What happens in GRPO (stage 4) — in plain words

GRPO (Group Relative Policy Optimization) is reinforcement learning *without* a separate "judge" model. For each question the model produces **several attempts**; because these questions have a **verifiable correct answer**, each attempt can be auto-graded. Attempts that score **above the group average** are reinforced; below-average ones are discouraged. A **reference model** and **reward normalization** keep the updates stable so the model improves without "drifting". The payoff: noticeably better physical reasoning.

Key experimental results

Cosmos-Reason1: bigger model & RL both help

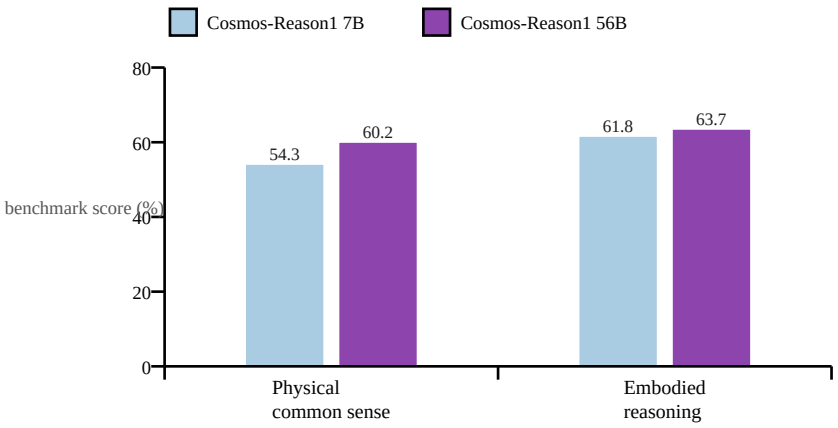


Figure 15. The 56B model beats the 7B on both physical-common-sense and embodied-reasoning benchmarks. Embodied reasoning improved by ~+10–11 points over the next-best prior model. **Source note:** numbers are from NVIDIA's Cosmos-Reason1 research page (released 7B checkpoint). An earlier arXiv v1 of the paper reported an *8B* model with somewhat different scores — see the verification page.

Reinforcement learning (GRPO) lifts reasoning further

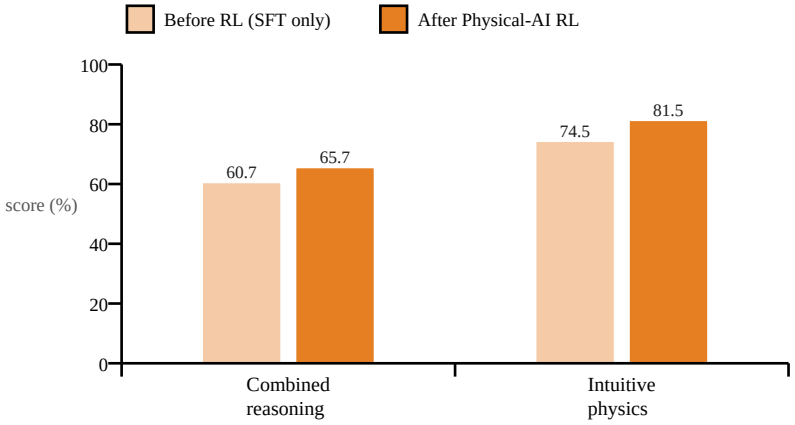


Figure 16. Adding the GRPO reinforcement-learning stage raised the 7B model by +5.0 points (combined reasoning) and +7.0 points (intuitive physics) — evidence that post-training, not just size, drives physical reasoning. **Source note:** research-page figures; arXiv v1 reports different magnitudes (e.g. intuitive physics 65.7→68.7). The *direction* — RL helps — is consistent across both.

Generation-quality results (Cosmos 2.5 & 3)

- **Cosmos 2.5** post-trained **14B** scores **0.810** on PAI-Bench Image2World (matching the strong Wan2.2-A14B baseline).
- **Cosmos 2.5** autonomous-driving multiview model: up to **2.3×** better FVD/FID (video realism metrics) versus its predecessor.
- **Cosmos 3** ranks **#1 among open models** on Artificial Analysis, Physics-IQ, PAI-Bench and R-Bench (world generation); RoboLab and RoboArena (action policy); and VANTAGE-Bench and TAR (vision understanding).

Note: FVD/FID measure how realistic/consistent generated video is (lower is better, so a 2.3× improvement is large). PAI-Bench scores world-generation quality (higher is better).

Verification & source audit

Every quantitative claim in this guide was cross-checked against primary sources (NVIDIA papers, research pages, and the official Cosmos 3 release). Of 21 checked claims, **19 verified** against a primary/official source; **2 carry caveats** (marked "!"). No factual errors were found in any claim that could be checked against a primary source.

#	Claim in this guide	Checked against	Verdict
1	Cosmos 1 data mix: 20/16/16/11/10/8/8/4/7%	arXiv 2501.03575	OK — exact
2	~100M clips (10^8) from ~20M hours of video	arXiv 2501.03575	OK — exact
3	Diffusion 7B = 28L / 4096 / 32 heads; 14B = 36L / 5120 / 40	Table 11	OK — exact
4	AdaLN-LoRA rank 256; LR 2^-15 / 2^-16; context 56,320	arXiv 2501.03575	OK — exact
5	Autoregressive 4B / 12B (+ 5B / 13B Video2World)	arXiv 2501.03575	OK — exact
6	Trained on ~10,000 H100 GPUs for ~3 months	arXiv 2501.03575	OK — exact
7	Tokenizer up to 12× faster; sparse attn (NATTEN) 50→98% sparsity, 1.7–2.6×	arXiv 2501.03575; arXiv 2504.16922	OK
8	Cosmos 2: 0.6B/2B/14B; 480/704p; flow matching in 2.5	GitHub + NVIDIA blog	OK
9	Cosmos 2.5: flow; Reason1 encoder; 2B/14B; 200M clips; 30s; PAI-Bench 0.810; 2.3× FVD/FID	NVIDIA research page	OK
10	GRPO post-training; 4-stage pipeline; verifiable rewards	arXiv 2503.15558	OK
11	Cosmos-Reason1 = 7B & 56B with the Fig 15–16 scores	Research page vs arXiv v1	CAVEAT — see below
12	Cosmos 3: mixture-of-transformers omnimodel; Super/Nano/Edge; #1 on open leaderboards	NVIDIA newsroom (Jun 2026)	OK
13	Cosmos 3 data = 20T tokens, ~1B images, ~400M videos	Press coverage (not NVIDIA)	CAVEAT — see below
14	Platform families: Predict / Transfer / Reason + Tokenizer / Curator / RL / Guardrails	Cosmos docs + DeepWiki	OK
15	Predict 1 variants (diffusion 7/14B, AR 4/12B, 5/13B V2W, upsampler, decoder)	arXiv 2501.03575 + GitHub	OK
16	Transfer = (Multi-)ControlNet on Predict; controls depth/edge/seg/LiDAR/HD-map	Cosmos docs + GitHub	OK
17	Transfer 2.5 built on Predict 2.5; 2B is 3.5× smaller than Transfer1-7B; control blocks distributed 1 per 7 (vs at start)	NVIDIA research page	OK — exact
18	Reason 1 = hybrid Mamba-MLP-Transformer; 16K context; SFT + RL	arXiv 2503.15558 (html)	OK — exact
19	Reason 2 = 2B/8B/32B; Qwen3-VL backbone (Reason1 was Mamba-hybrid); 256K context; OCR + 2D/3D localization; #1 open	HF Reason2 model card	OK — exact
20	Cosmos 3 = two-tower MoT (Nano 16B = 8+8, Super 64B = 32+32); ViT/VAE encoders; 3D mRoPE	NVIDIA/HF Cosmos 3 blog	OK
21	Cosmos 3 reasons + generates in ONE unified forward pass; separate weights, joint attention (vs the old 2-model pipeline). Causal/bidirectional masking is the typical pattern, not NVIDIA-confirmed.	NVIDIA/HF + dev blog	OK (masking inferred)

Table 14. Claims audit. Green = verified against a primary source; amber = caveat.

The two caveats, in detail

- **! Cosmos-Reason1 size & scores (claim 11):** NVIDIA's research page describes a released **7B** model with the scores plotted in Figures 15–16. The **arXiv v1** of the paper instead describes an **8B** model and reports different numbers (e.g. physical common sense 52.3% vs 54.3%; intuitive-physics RL gain 65.7→68.7 vs 74.5→81.5). This is a paper-revision difference. This guide uses the **research-page (7B)** figures because that matches the publicly released checkpoint — but treat the exact decimals as version-dependent. The qualitative conclusions (bigger model helps; RL helps) hold

in both.

- **! Cosmos 3 data scale (claim 13):** the “20 trillion tokens / ~1B images / ~400M videos” figures come from **press coverage** of the launch. NVIDIA's own newsroom release only says “billions of samples across text, image, video, sound and action.” Treat the precise counts as **unconfirmed by NVIDIA**.

Bottom line: the guide is accurate where primary sources exist. The only soft spots are (a) decimal-level benchmark numbers for Cosmos-Reason1, which differ between paper versions, and (b) Cosmos 3's exact data counts, which NVIDIA has not officially published.

Putting it all together

The four generations trace a clear arc:

- **Cosmos 1 (Toolbox):** two engines (diffusion + GPT), a strong causal tokenizer, T5 text, 9-category data, fully documented hyperparameters. Maximum flexibility.
- **Cosmos 2 (Polish):** diffusion-only, sparse attention for up to 2.6× speed, action-conditioned post-training. Same idea, executed better.
- **Cosmos 2.5 (Unify):** one flow-based model replaces three; Cosmos-Reason1 reads the prompt; 200M clips; 30s multi-camera video; RL + model-merge post-training.
- **Cosmos 3 (Leap):** mixture-of-transformers omnimodel; reasons before generating; adds audio and actions; ~20T-token training; Super/Nano/Edge tiers; #1 open model on many leaderboards.

The single sentence to remember

Cosmos went from **“many tools that dream video”** (1) → **“the best tool, polished”** (2) → **“one smart unified tool”** (2.5) → **“a reasoning, seeing, hearing, acting omnimodel”** (3).

Test your understanding

Use these to check you really absorbed the ideas (answers on the next page). If you can answer in a sentence or two, you've got it.

Basics

1. In one sentence, what is a “World Foundation Model”, and how is it different from a language model?

2. Cosmos is “a platform, not one model.” Name the three core families and what each one does.

The two engines (diffusion vs autoregressive)

3. What is the single most important factor that separates the diffusion engine from the autoregressive engine?

4. Explain “bidirectional” vs “causal next-token” attention in your own words — and say which engine uses which.

5. Why does the autoregressive engine need a “diffusion decoder”, while the diffusion engine does not?

6. Which engine is the natural fit for real-time, streaming world simulation — and why does that connect to how the physical world actually works?

7. Why does “error drift” hurt the autoregressive engine but not the diffusion engine in the same way?

Tokenizer & techniques

8. The tokenizer has two flavours (continuous and discrete). Which feeds diffusion, and which feeds autoregressive?

9. In easy terms, what does sparse attention (NATTEN) do, and roughly what speedup did it give Cosmos 2?

10. What problem does flow matching solve versus diffusion, and what is the “straight vs wandering path” idea?

Platform & generations

11. Was each new generation just “a bigger model on more data”? Give one real novelty for 1→2, 2→2.5, and 2.5→3.

12. The families don't version in lockstep. Which generation was essentially a “Predict-only” refresh?

13. What is Transfer's job, and how does it differ from Predict?

Cosmos 3 (advanced)

14. Does Cosmos 3 reason first and then generate (staged), or both at once? Explain “separate weights but joint attention.”

15. How does Cosmos 3 fuse five modalities (incl. audio) into one model? Name the three ingredients, and map the old families onto its two towers.

16. Are Cosmos Reason 1 and Reason 2 the same model? What backbone does each use — and what's the honest caveat about “Reason 2-2B beats Reason 1-7B”?

Answer key

1. A model that learns from video to predict what happens next in the **physical world** (how things move/collide), instead of predicting the next **word** like a language model. Goal: simulate the world for Physical AI.

2. Predict = generate the future world (video); **Transfer** = controllable sim→real from structure maps; **Reason** = a VLM that understands and decides. (+ Tokenizer/Curator/RL/Guardrails tools.)

3. How and in what order they generate: diffusion = holistic / parallel / continuous (refine the whole clip at once); autoregressive = causal / sequential / discrete (predict the next token from the past).

4. Bidirectional (diffusion): a token sees tokens before AND after it (fill-in-the-blank). **Causal next-token** (autoregressive): a token sees only the past and predicts the next (autocomplete / time).

5. Autoregressive uses heavy discrete compression that adds artifacts, so a small diffusion decoder cleans its output. Diffusion already works in continuous space, so it needs no fix.

6. Autoregressive — it builds the future from the past one step at a time, which mirrors time's arrow and how a robot acts step by step, so it can stream/extend indefinitely.

7. Autoregressive feeds its own (possibly wrong) outputs back in, so errors compound over time. Diffusion refines the whole clip together, so there's no left-to-right snowball.

8. Continuous tokens → diffusion; **discrete** (FSQ, 64k vocab) tokens → autoregressive.

9. Each patch attends only to nearby patches (a local window) instead of all patches, dropping up to ~98% of connections → **up to 2.6× faster** at 720p.

10. Diffusion follows a curved, many-step path from noise to video; flow matching learns a near-**straight** path you can cover in **far fewer, bigger steps** → faster, cleaner.

11. No. 1→2 = sparse attention (≈2.6×) + action conditioning; 2→2.5 = task unification + flow matching + Reason1 encoder + RL; 2.5→3 = mixture-of-transformers + audio/action modalities.

12. Cosmos 2 — mainly a Predict upgrade (Transfer and Reason didn't get a “2” then).

13. Transfer turns **structured inputs** (depth/segmentation/LiDAR/HD-map) into photorealistic video (sim→real) via ControlNet on top of Predict; Predict generates more freely from text/image/video.

14. Both at once — one unified forward pass. The two towers have their own weights (specialised) but share **joint attention** at every layer, so reasoning and generation are **coupled, not deferred**. (The reasoner can also run alone as a VLM.)

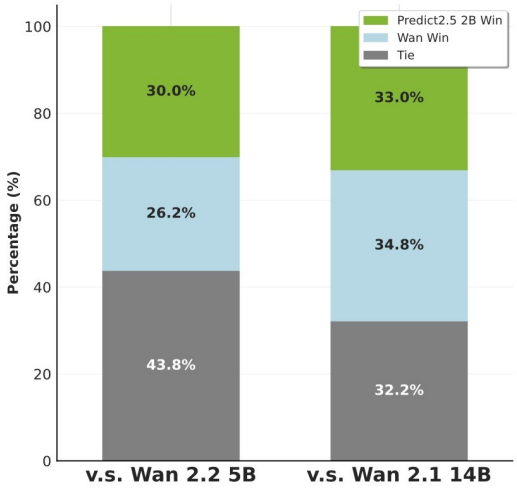
15. (i) a dedicated encoder per modality → a shared space; (ii) one token sequence + global attention aligned by 3D mRoPE; (iii) Mixture-of-Transformers (own weights per modality, shared attention). Towers: Reason → reasoner; Predict + Transfer → generator.

16. No — different models. Reason 1 = NVIDIA hybrid Mamba-MLP-Transformer (7B/56B); Reason 2 = post-trained from Qwen3-VL (2B/8B/32B). The 2B>7B read is directionally right, but the benchmarks (PAI-Bench versions) differ, so it isn't an exact same-test comparison.

Appendix A — Example results (from NVIDIA's pages)

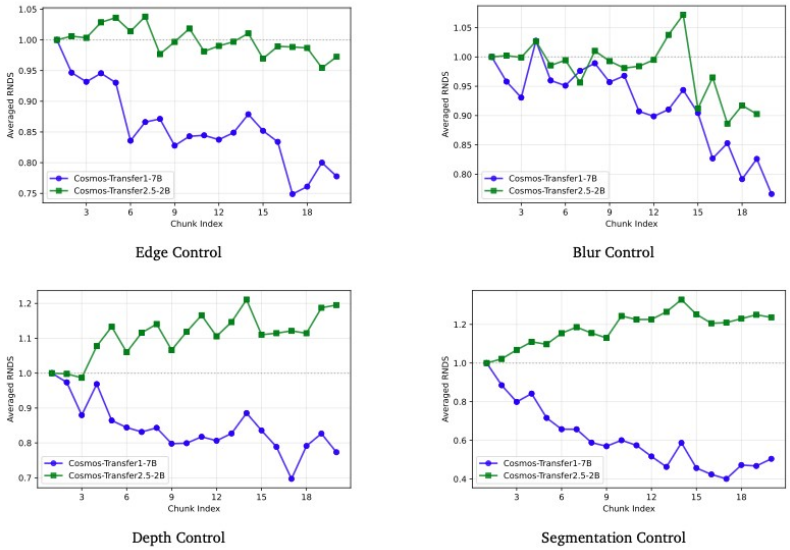
These are real result figures and example inputs taken from NVIDIA's official Cosmos pages, reproduced here for educational reference. © NVIDIA; see the link under each image.

A1 · Cosmos Predict 2.5 — human-preference win rate vs. Wan baselines



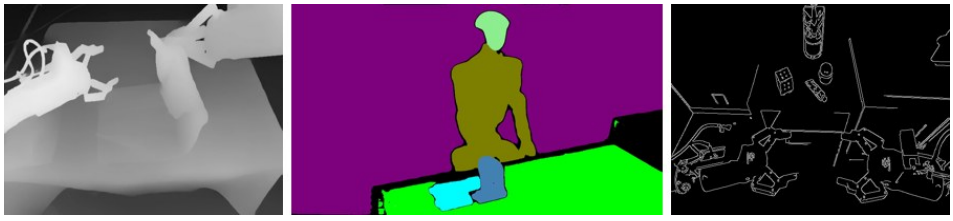
Predict 2.5 (2B) “win / tie / lose” against Wan 2.2 (5B) and Wan 2.1 (14B) in human preference — a small model competitive with much larger baselines. Source: research.nvidia.com/labs/cosmos-lab/cosmos-predict2.5/

A2 · Cosmos Transfer 2.5 (2B) vs. Transfer 1 (7B) — quality across long videos



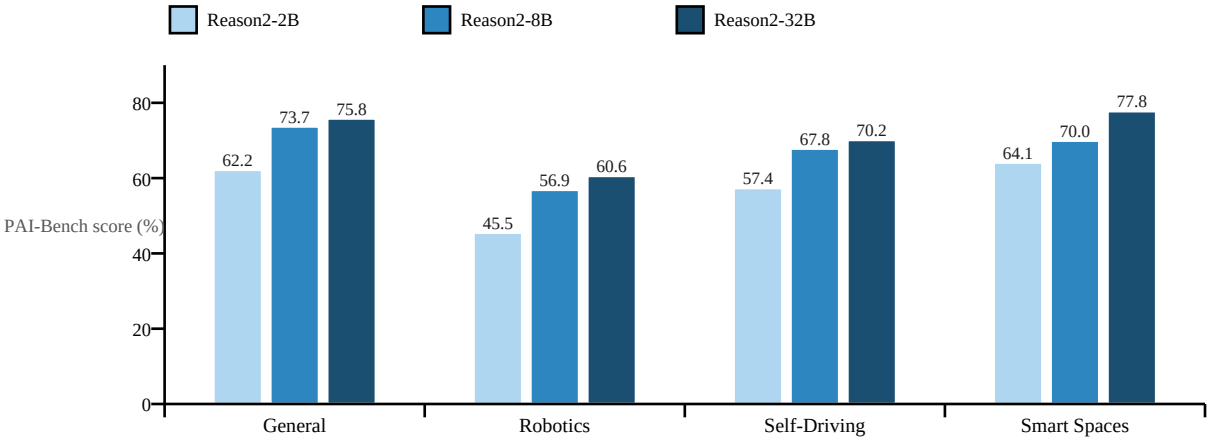
Averaged RNDS (higher = better) across video “chunk index” for Edge / Blur / Depth / Segmentation control. Transfer 2.5-2B (green) stays high while Transfer 1-7B (blue) degrades over long sequences — the 3.5× smaller model is also better. Source: research.nvidia.com/labs/cosmos-lab/cosmos-transfer2.5/ (Figure 8)

A3 · What Cosmos Transfer ingests — example control inputs



Left→right: depth, segmentation and edge control maps. Cosmos Transfer turns structured inputs like these into photorealistic video (sim-to-real). Source: research.nvidia.com/labs/cosmos-lab/cosmos-transfer2.5/

A4 · Cosmos Reason 2 — Physical AI Bench by model size



Reason 2 (built on a Qwen3-VL backbone) across the four Physical AI Bench domains. Even the small 2B (62.21 General) is competitive with the previous-generation Reason 1-7B (~56% on its older benchmark) — a real efficiency leap, though the benchmarks differ in version. Source: huggingface.co/nvidia/Cosmos-Reason2-8B (model card).

Appendix B — References & further reading

Primary papers

• **Cosmos World Foundation Model Platform for Physical AI (Cosmos 1)**
https://arxiv.org/abs/2501.03575

• **Cosmos-Reason1: From Physical Common Sense to Embodied Reasoning**
https://arxiv.org/abs/2503.15558

• **Mixture-of-Transformers: A Sparse, Scalable Multi-Modal Architecture**
https://arxiv.org/abs/2411.04996

• **Generalized Neighborhood Attention (sparse attention / NATTEN)**
https://arxiv.org/abs/2504.16922

NVIDIA research & model pages

• **Cosmos-Predict2.5**
https://research.nvidia.com/labs/cosmos-lab/cosmos-predict2.5/

• **Cosmos-Transfer2.5**
https://research.nvidia.com/labs/cosmos-lab/cosmos-transfer2.5/

• **Cosmos 3 technical report**
https://research.nvidia.com/labs/cosmos-lab/cosmos3/technical-report.pdf

• **Cosmos model families (docs / DeepWiki)**
https://deepwiki.com/nvidia-cosmos/cosmos-cookbook/3-cosmos-model-families

Blogs & announcements

• **HF: Cosmos Predict 2.5 & Transfer 2.5**
https://huggingface.co/blog/nvidia/cosmos-predict-and-transfer2-5

• **HF: Cosmos Reason 2 brings advanced reasoning**
https://huggingface.co/blog/nvidia/nvidia-cosmos-reason-2-brings-advanced-reasoning

• **HF: Welcome Cosmos 3 for Physical AI**
https://huggingface.co/blog/nvidia/cosmos-3-for-physical-ai

• **NVIDIA blog: Develop Physical AI with Cosmos 3**
https://developer.nvidia.com/blog/develop-physical-ai-reasoning-world-and-action-models-with-nvidia-cosmos-3/

• **NVIDIA newsroom: Cosmos 3 launch (Jun 2026)**
https://nvidianews.nvidia.com/news/nvidia-launches-cosmos-3-the-open-frontier-foundation-model-for-physical-ai

GitHub

• **cosmos-predict1 / predict2 / predict2.5**
https://github.com/nvidia-cosmos

• **cosmos-transfer1 / transfer2.5**
https://github.com/nvidia-cosmos

• **cosmos-reason1 / reason2**
https://github.com/nvidia-cosmos